Defect Resolution During Product Development: Two Approaches from Industry

David Vo
McGill University
Montreal, Canada
david.vo2@mail.mcgill.ca

David Zhou
McGill University
Montreal, Canada
david.zhou3@mail.mcgill.ca

Abstract

This paper reports on defect resolution practices during product development as described by Eric Deng and Nathan Audegond in separate interviews. Eric worked as a Developer Intern at Ericsson Canada from May to December 2024, building a conversational AI chatbot for the company's QA team. Nathan worked as a Test Automation Developer Intern at Intact Financial Corporation during Summer 2025, building a regression testing tool for an insurance module migration. Both interviewees described a typical example, a success, and a failure from their experience with defect resolution. The paper compares and contrasts the two approaches, identifies the strengths and weaknesses of each, and suggests process improvements for both teams.

Keywords

defect resolution, software testing, agile, code review, CI/CD, QA automation

1 Introduction

Finding and fixing bugs during product development is one of the most routine and important parts of software engineering. How a team handles that process, how defects are discovered, tracked, communicated, and resolved, shapes the quality of the software and the experience of the people building it. Different organizations have very different approaches to this, and neither a completely informal nor a completely rigid process is always the right answer.

This paper looks at defect resolution during development as described by two McGill Software Engineering co-op students, Eric Deng and Nathan Audegond, based on interviews we conducted with each of them [2][3]. Eric worked at Ericsson Canada on an internal QA tooling project. Nathan worked at Intact Financial Corporation on a customer-facing insurance automation platform. Both gave us three concrete examples from their experience: a typical case, a success, and a failure. The goal of this paper is to compare how each team approached defect resolution, identify what each did well and where things fell short, and suggest some concrete ways each process could be improved.

1.1 Eric Deng

Eric Deng is a third-year Software Engineering student at McGill who completed an eight-month co-op at Ericsson Canada from May to December 2024 [4]. He was placed on Team Dragon, the QA team within the Global Network Platform organization, which is responsible for testing the 5G network API platform that Ericsson sells to telecom providers. The team used an in-house testing framework called TESU to run tests against GNP's microservices. Eric's main project was a conversational AI chatbot built on the RASA framework, which let testers trigger test cases through plain language commands rather than manual shell scripts. He was the sole developer on the chatbot for the entire duration of his internship.

1.2 Nathan Audegond

Nathan Audegond is also a Software Engineering student at McGill who completed a four-month co-op at Intact Financial Corporation during Summer 2025 [5]. It was his second co-op internship, following a previous placement at Ericsson. At Intact he worked on ContractUBI, a project migrating the Usage-Based Insurance module of Intact's insurance platform from a legacy XML architecture to a modern JSON-based system built on Java Spring Boot. His main deliverable was UBI Tools, a full-stack web application built with Angular and Node.js that automated regression testing for the UBI module's REST APIs and let QA engineers compare test results across different builds to catch regressions.

1.3 More Information

The rest of this paper is based on the interview transcripts from our conversations with Eric and Nathan [2][3]. Both interviewees were asked to describe their defect resolution process in general, and then to walk through a typical case, a success story, and a failure from their own experience.

2 Describing the Approaches

2.1 Eric's Approach at Ericsson

Eric's defect resolution process was almost entirely self-directed and informal. Because he was the only developer on the chatbot, there was no one else whose code he was integrating with on a daily basis, and there was no formal pipeline that would run tests automatically when he made changes. His main method of finding defects was manual testing: after making a change to the chatbot's RASA configuration files and retraining the model, he would run through test conversations in the browser and observe whether the chatbot responded correctly. If something was wrong, he would try to reproduce the issue consistently, isolate which layer was responsible (intent classification, dialogue rules, or the action server logic), make a fix, retrain, and test again. He would report what he found and fixed at the next morning's stand-up, where his supervisor Daniel or his mentor Yash might offer feedback if something seemed off [2].

There was no formal bug tracking system for Eric's chatbot work specifically. Defects were caught, fixed, and mentioned at stand-up, but not logged in Jira unless they were significant enough to become their own story or improvement ticket. The team's view was that since the chatbot was not on the critical path of GNP's

production environment, the overhead of filing a Jira ticket for every bug was not worth it [2].

2.2 Nathan's Approach at Intact

Nathan's defect resolution process was built into a much more structured development workflow. Every time Nathan pushed a change to a feature branch on GitHub, a Jenkins CI/CD pipeline triggered automatically and ran the unit and integration tests for UBI Tools. If any tests failed, the build broke and Nathan was notified immediately. This meant the first line of defect detection was automated and fast, typically surfacing issues within minutes of a commit [3].

If the automated tests passed but Nathan suspected a behavioral issue, he would open a pull request and request a code review from a teammate. Reviews frequently caught edge cases in the comparison logic or problems with how API responses were being handled that Nathan had not tested. Only after peer review and SonarQube static analysis approval could a PR be merged. For new features, a separate QA engineer also ran through the acceptance criteria from the Jira ticket manually, independent of Nathan. When a defect was found, it was logged as a bug in Jira with a description, reproduction steps, and a priority level, then picked up in the current or next sprint [3].

3 Comparing Approaches

3.1 Similarities

Both Eric and Nathan worked in Agile Scrum environments and used Jira as their task management tool. Both described a development process where work was broken into small, iterative chunks and where stand-up meetings served as the main forum for surfacing blockers and communicating about issues [2][3]. In both cases, defects that were significant enough to need future attention were eventually logged in Jira as stories or improvement tickets, even if the initial discovery happened informally.

Both interviewees also emphasized the importance of being able to reproduce a defect reliably before attempting to fix it, and both described debugging as a process of narrowing down through layers of the system to find the root cause. This reflects a general principle that defect resolution is more effective when the diagnosis step is taken seriously rather than jumping straight to a fix [1].

3.2 Differences

The most significant difference is how defects were first discovered. At Ericsson, defects were found manually by Eric himself during his own testing, with no automated safety net between writing code and discovering a problem. At Intact, the Jenkins CI/CD pipeline ran tests automatically on every push, meaning defects were surfaced by the system rather than depending solely on Nathan noticing something was wrong during a manual test session [3]. As Pressman notes, automation in the testing process removes the dependency on human memory and attention, and allows defects to be caught consistently at the point where they are introduced [6].

The second major difference is in how defects were tracked. At Ericsson, most bugs were handled informally with no written record unless they became large enough to warrant a Jira ticket. At Intact, even minor bugs discovered during development were logged in Jira with enough context for someone else to understand them. McConnell argues that defect tracking data is itself a valuable engineering asset, because patterns in where defects cluster can reveal systemic problems in the design or process that would otherwise go unnoticed [7]. Ericsson's informal approach lost that information.

The third difference is in peer involvement. At Ericsson, Eric was alone, and the only other person who looked at his code was his mentor Yash during informal stand-up discussions. At Intact, every change went through at least one peer review and a dedicated QA engineer also checked the behavior against acceptance criteria. Myers writes that code inspections are one of the highest-return testing practices available, and that independent testing by someone other than the author is significantly more effective at finding defects [8]. This gap is clear when comparing the two experiences.

4 Strengths of Eric's Approach

4.1 Fast Individual Iteration

One practical strength of Eric's process was how quickly he could move through individual fix-and-test cycles. Because there was no pipeline to wait for and no review queue to join, Eric could make a change and see the result in the browser within a few minutes of retraining the model. For an experimental project where the chatbot's behavior was being shaped and refined continuously, this tight feedback loop was genuinely useful. It allowed him to try different dialogue designs quickly and discard ones that did not work [2].

4.2 Documentation as Quality Practice

Another strength was that Eric was required to produce formal documentation as a deliverable, a deployment guide for QA testers and a development guide for future TESU contributors. This is worth noting as a quality practice because writing documentation forces the author to examine their own design decisions carefully. Eric mentioned that writing the development view document helped him notice inconsistencies in how he had structured some of the chatbot's action functions, which he then went back and fixed. Good documentation requirements can serve as a lightweight form of design review [4].

5 Weaknesses of Eric's Approach

5.1 No Automated Defect Detection

The most significant weakness was the complete absence of automated testing. Every defect in the chatbot was caught only when Eric manually ran through a conversation and happened to observe the incorrect behavior. This is highly dependent on the developer thinking to test the right scenario at the right time. In Eric's failure example, a defect in the YAML parsing logic for tc_parameters.yml went unresolved for a full week not because it was hard to fix, but because it took that long to recognize that the approach was wrong [2]. An automated test that validated the parser against a representative sample of the YAML file's structure could have flagged the problem much earlier.

5.2 No Formal Defect Logging

Because defects were not logged in Jira, there was no record of what had gone wrong over the course of the project. Eric reflected that he could recall having four or five bugs that all traced back to incorrect assumptions about the structure of TESU's configuration files, but since he had not written them down, he only recognized the pattern in retrospect. A lightweight defect log would have made this pattern visible earlier and prompted a more systematic approach to validating assumptions before implementation [2].

5.3 No Peer Review

Without any form of code review, Eric's defects were limited to what he himself could spot. His mentor Yash had relevant knowledge that would have been useful earlier, as demonstrated by the tc_parameters.yml failure, where Yash immediately understood why the file was unusable once Eric showed it to him. That conversation happened after a week of wasted effort rather than before it [2]. More broadly, code review does not just catch bugs in the code being reviewed; it also transfers knowledge about the system from the reviewer to the author, which reduces the likelihood of similar mistakes in the future. A developer working in isolation misses both of those benefits.

6 Improvements for Eric's Approach

6.1 Lightweight Code Reviews

The most practical improvement for Ericsson's team would be to make code review a regular part of the workflow, even informally. This does not need to be a formal pull request process for every commit. A reasonable approach for a small team with one developer on a project would be a weekly review session where the developer walks the mentor through recent changes and the mentor asks questions. McConnell documents that code inspection is one of the most cost-effective defect detection practices available, consistently finding bugs that automated tests miss because reviewers bring different mental models to the code [7]. Even 30 minutes a week with Yash would have caught many of the issues Eric spent days debugging alone.

6.2 Automated Tests for Core Logic

Eric should have written unit tests for the core Python functions in actions.py, particularly the ones responsible for parsing TESU configuration files and routing chatbot actions. These functions were the layer where the most significant defects appeared, and they were also the most straightforward to test in isolation. A small suite of unit tests covering the expected input formats would not have been expensive to write and would have caught the tc_parameters.yml parsing problem automatically when the structure of the file did not match expectations. Pressman describes unit testing as the first and most efficient line of defence against defects, because the cost of finding a bug at the unit level is a fraction of finding it during integration or manual testing [6].

7 Strengths of Nathan's Approach

7.1 Fast Automated Feedback

The Jenkins CI/CD pipeline at Intact gave Nathan rapid, consistent feedback on every change. When a build failed, Nathan knew within minutes that something was wrong, and the test output usually pointed directly to the problem. This dramatically reduced the time between introducing a defect and finding it. For a project where the codebase was growing quickly and multiple features were being built in parallel, having that automatic safety net prevented many small bugs from compounding into larger integration problems [3].

7.2 Peer Review as a Second Line of Defence

The mandatory code review process at Intact caught defects that the automated tests did not cover. Nathan gave several examples during the interview of reviewers flagging edge cases in the comparison logic or unhandled API responses that he had not thought to test. This is exactly the kind of complementary defect detection that Myers describes: automated tests are efficient for repeatable, well-defined scenarios, while human reviewers are better at recognizing logic that looks structurally sound but is semantically wrong [8].

7.3 Formal Defect Tracking

When a bug was reported at Intact, it was logged in Jira with context, steps to reproduce, and a priority level. This meant every defect was visible to the whole team, could be prioritized alongside other work, and left a trace in the project history. The floating point precision bug, for example, was reported informally at a stand-up but then immediately turned into a Jira ticket that was picked up in the same sprint [3]. That traceability is valuable both for the current team and for anyone maintaining the project in the future. When Nathan added the rounding logic to fix the bug, the commit message referenced the Jira ticket, so any future developer reading the code would be able to find the original report and understand the context behind the change. This kind of documented history is something that informal bug handling, like Eric's approach at Ericsson, cannot provide.

8 Weaknesses of Nathan's Approach

8.1 Defects Slipping Through Late in the Process

Despite the strong automated and peer review processes, Nathan's most significant defect, the floating point precision issue in the comparison logic, went undetected for several weeks because his test coverage did not include the range of endpoint response types that were present in the real system. The bug was eventually found informally by a developer using the tool, not by any part of the formal process [3]. This points to a gap between having testing infrastructure and using it systematically. The CI pipeline ran tests, but the tests themselves were only as good as the scenarios Nathan thought to cover.

8.2 Delayed Pull Request Reviews

Nathan mentioned that pull requests would sometimes sit for a day or two before a reviewer was available, which occasionally blocked his progress on a feature that depended on the previous change

being merged. While the review process was sound, the lack of any expectation around how quickly reviews should happen made the pipeline unpredictable. In a sprint planning context, not knowing when a PR will be reviewed makes it harder to commit to story completion within a given sprint [3].

9 Improvements for Nathan's Approach

9.1 Input Space Partitioning for Test Coverage

The floating point defect revealed that Nathan's test cases were implicitly designed around integer-valued fields, which is the domain he happened to be working in. A more systematic approach would be to explicitly identify the different categories of data each feature needs to handle, integers, floats, strings, null values, nested objects, and write at least one test case for each category before calling a feature complete. Sommerville describes this approach as equivalence partitioning: dividing the input space into classes of values that are likely to behave differently, and writing tests that cover each class [1]. Applying this discipline to the comparison feature would almost certainly have included a floating point test case, and the bug would have been found before it reached any real users.

9.2 Review Turnaround Expectations

To address the pull request delay problem, Intact's team should agree on a maximum response time for reviews. A commonly used standard is that a reviewer should provide at least an initial response within one business day of a PR being opened. This does not mean the full review needs to be completed in a day, but it prevents reviews from being silently deferred indefinitely. Forsgren, Humble, and Kim's research on high-performing engineering teams shows that short lead times for code changes are one of the strongest predictors of both software delivery performance and developer satisfaction [9]. Establishing a shared expectation around review turnaround would be a small process change with a meaningful impact on sprint predictability.

10 Acknowledgement

We would like to thank Eric Deng and Nathan Audegond for taking the time to share their experiences. Their honest accounts of both the things that worked and the things that did not gave us a much clearer picture of how defect resolution actually works in practice, and made this paper possible.

11 References

References

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
[2] D. Zhou (Interviewer) and E. Deng (Interviewee), "Interview on defect resolution during product development at Ericsson Canada," February 2026. Transcript: *EricDengInterview.md*.
[3] D. Vo (Interviewer) and N. Audegond (Interviewee), "Interview on defect resolution during product development at Intact Financial Corporation," February 2026. Transcript: *NathanAudegondInterview.md*.
[4] E. Deng, "Internship report: Developer Intern, Ericsson Canada," internal report, December 2024.
[5] N. Audegond, "Internship summary: Test Automation Developer Intern, Intact Financial Corporation," internal report, Summer 2025.
[6] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.
[7] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*, 2nd ed. Microsoft Press, 2004.
[8] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, 2011.
[9] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.